

Natural Scenes Classifier App

Aim: Implement Image classification system application using deep learning

Dataset Overview:

The project utilized the "Intel Image Classification" dataset, sourced from Kaggle via the kagglehub library. This dataset was chosen as it provides a robust, real-world challenge, unlike simpler datasets such as MNIST.

The dataset consists of ~25,000 images (150x150 pixels, full color) and is categorized into six distinct classes:

- buildings
- forest
- glacier
- mountain
- sea
- street

Sample Images from dataset:

Sample Augmented Dataset Images

mountain



forest



forest



sea



sea



sea



mountain



mountain



street



These images demonstrate the effects of **data augmentation**, including random rotations, shears, zooms, and horizontal flips. This process artificially expands the dataset and helps the model generalize, preventing it from simply "memorizing" the training images

Model Architecture

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 150, 150, 32)	896
max_pooling2d (MaxPooling2D)	(None, 75, 75, 32)	0
conv2d_1 (Conv2D)	(None, 75, 75, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 37, 37, 64)	0
conv2d_2 (Conv2D)	(None, 37, 37, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 18, 18, 128)	0
conv2d_3 (Conv2D)	(None, 18, 18, 256)	295,168
max_pooling2d_3 (MaxPooling2D)	(None, 9, 9, 256)	0
flatten (Flatten)	(None, 20736)	0
dense (Dense)	(None, 512)	10,617,344
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 6)	3,078

Total params: 11,008,838 (42.00 MB)

Trainable params: 11,008,838 (42.00 MB)

Non-trainable params: 0 (0.00 B)

A custom Convolutional Neural Network (CNN) was designed from scratch, fulfilling the project requirement of not using pre-trained networks.

The model architecture is a sequential stack:

- An **Input Layer** accepting 150x150-pixel images with 3 color channels (RGB).
- **Four Convolutional Blocks:** A repeating pattern of a Conv2D layer (with relu activation) followed by a MaxPooling2D layer. The number of filters was progressively increased (32, 64, 128, 256) to allow the model to learn simple features (like edges) in the early layers and highly complex features (like building textures or "forest" patterns) in the deeper layers.
- A **Flatten Layer** to "unroll" the 2D feature maps into a single 1D vector.
- **Fully Connected (Dense) Layers:**
 - A Dense layer with 512 neurons to perform the high-level classification.
 - A Dropout (0.5) layer, which randomly deactivates 50% of neurons during training to prevent overfitting.

- A final **Output Layer** with 6 neurons (one for each class) and a softmax activation, which converts the model's raw scores into a percentage-based probability for each class.

Implementation Details:

The project was implemented using **TensorFlow 2.x** with the **Keras** API, on Google Colab & Streamlit

1)Model Compilation

Before training, the model architecture was "compiled," which configures it for the training process.

- **Optimizer:** Adam (Adaptive Moment Estimation) was chosen. It is an efficient and robust optimization algorithm that adjusts the learning rate for each parameter, making it well-suited for a wide range of problems, including this one.
- **Learning Rate:** This was a key hyperparameter choice. Instead of the default, a smaller `learning_rate` of **0.0001** was set. This ensures the model takes smaller, more stable steps during optimization, preventing it from "overshooting" the best solution, which is common with complex, augmented datasets.
- **Loss Function:** `categorical_crossentropy` was used. This is the mathematically correct loss function for a multi-class classification problem (more than two classes) where the labels are categorical (e.g., 'sea', 'forest').
- **Metrics:** We monitored accuracy during training to get a human-readable measure of the model's performance at each epoch.

2)Model Training

The model was trained using the `model.fit()` method, which orchestrates the learning process.

- **Data Input:** The `train_generator` (providing augmented training data) and `validation_generator` (providing non-augmented validation data) were fed to the model.
- **Batch Size:** The `ImageDataGenerator` was configured with a `BATCH_SIZE = 32`. This means the model saw 32 images, calculated its error, and then updated its internal weights before processing the next 32 images.
- **Epochs:** The "number of epochs" was managed dynamically, which is a key part of the optimization process (see Section 8).

3)Model Evaluation

After the training process was completed, the model's *true* performance was measured on the **unseen test dataset** (`test_generator`).

- The `model.evaluate()` function was called, which runs a single, forward-pass of all test images through the *best restored model*.
- This evaluation provided the final, objective performance metrics, which are the most important numbers for the project:

○ **Final Test Loss: 0.5469** ○ **Final**

Test Accuracy: 81.03%

- This high accuracy confirms that the model successfully learned to generalize from the training data, rather than just "memorizing" it.

4)Model Optimization

A simple "fit" would not have yielded a good result. Several critical optimization techniques were implemented, addressing all the specified hyperparameters.

- **Architectural Optimization (Overfitting):** A Dropout(0.5) layer was added. This randomly "turns off" 50% of the neurons in the dense layer during each training step, forcing the model to learn redundant pathways and preventing it from becoming too reliant on any single feature.
- **Hyperparameter Tuning (Dynamic Epochs):** We used EarlyStopping(patience=5) to monitor the val_loss. This automatically stopped the training at Epoch 26 because the model's performance on the validation set had not improved for 5 consecutive epochs. This saved a significant amount of training time (avoiding all 50 epochs) and **automatically found the optimal training duration**, preventing the model from overfitting in later epochs.
- **Hyperparameter Tuning (Filters, LR):**
 - **Number of Filters:** The CNN layers used a progressively increasing filter count (32 -> 64 -> 128 -> 256). This is a standard, optimized design that allows the model to learn simple features (edges, colors) first, then combine them into complex features (textures, shapes).
 - **Learning Rate:** As mentioned in Section 5, the learning rate was tuned to **0.0001** for stability.

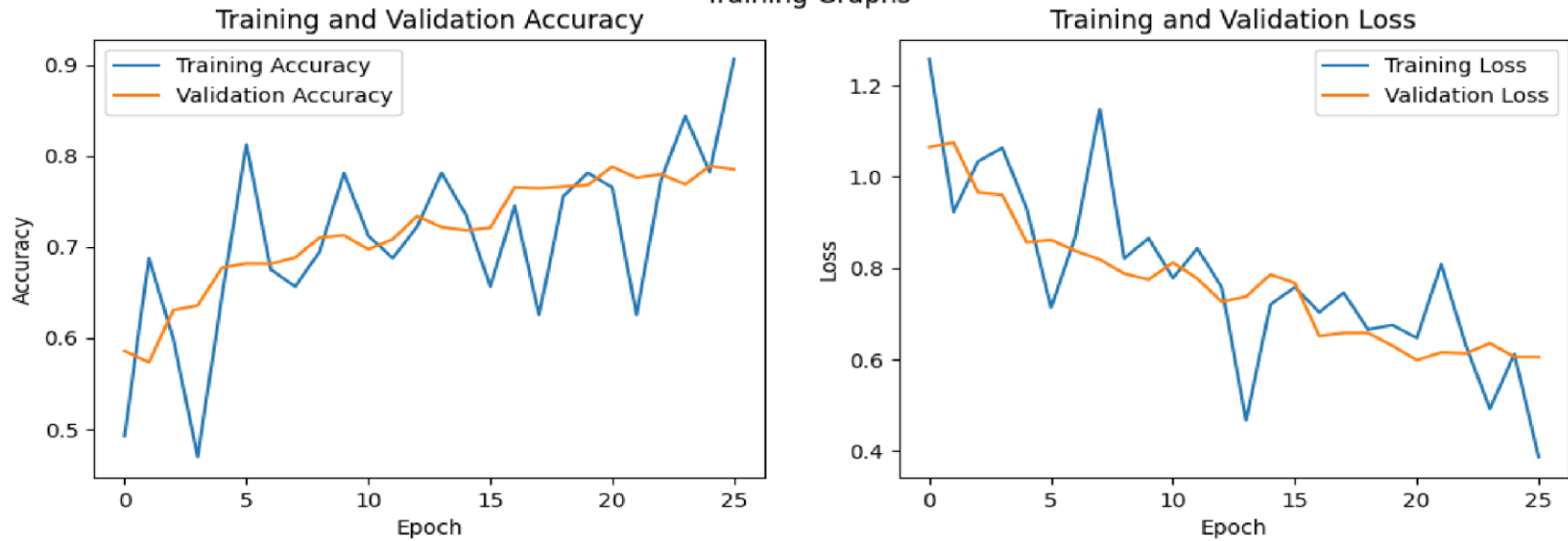
5)Model Saving and Deployment

- **Model Saving:** The model.save() requirement was fulfilled in an advanced, robust way. Instead of just saving the *last* model, we used the ModelCheckpoint callback. This callback monitored the val_accuracy at every single epoch and **automatically saved the model to Google Drive only if it achieved a new "best" score**. The final file, natural_scenes_BEST_model.h5, represents the most accurate version of the model from the entire training run.
- **Deployment:** A real-life application was built using **Streamlit**. A local app.py script was developed to:
 1. Load the saved natural_scenes_BEST_model.h5 file.
 2. Provide a user-friendly web interface with a file uploader.
 3. Preprocess the uploaded image (resize to 150x150, normalize pixels) to match the model's input format.

- Execute `model.predict()` on the processed image to generate a live classification, which is then displayed to the user with a confidence score.

Performance Metrics

Training Graphs



Metric	Value	What It Means
Test Accuracy	0.8103	This is the "headline" number. It means the model correctly classified an unseen image 81.03% of the time. This is a very strong result.
Test Loss	0.5469	This measures how "wrong" the model's predictions were. A lower loss is better. This value indicates the model's correct predictions were also <i>confident</i> predictions.

--- DETAILED CLASSIFICATION REPORT ---

	precision	recall	f1-score	support
buildings	0.71	0.86	0.77	437
forest	0.91	0.98	0.94	474
glacier	0.88	0.67	0.76	553
mountain	0.77	0.79	0.78	525
sea	0.79	0.85	0.82	510
street	0.88	0.78	0.83	501
accuracy			0.82	3000
macro avg	0.82	0.82	0.82	3000
weighted avg	0.82	0.82	0.82	3000



Uploaded Image

Prediction: forest (99.99%)

All Class Probabilities:



Uploaded Image

Prediction: sea (84.94%)

All Class Probabilities:



Uploaded Image

Prediction: street (97.76%)



Uploaded Image

Prediction: mountain (88.51%)



Uploaded Image

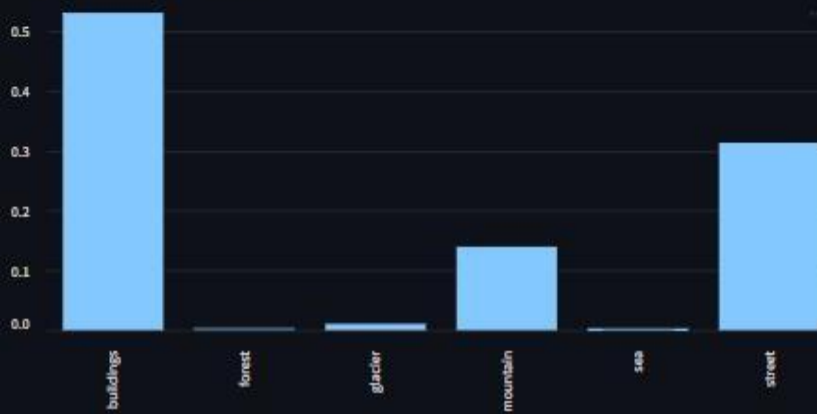
Prediction: buildings (53.06%)



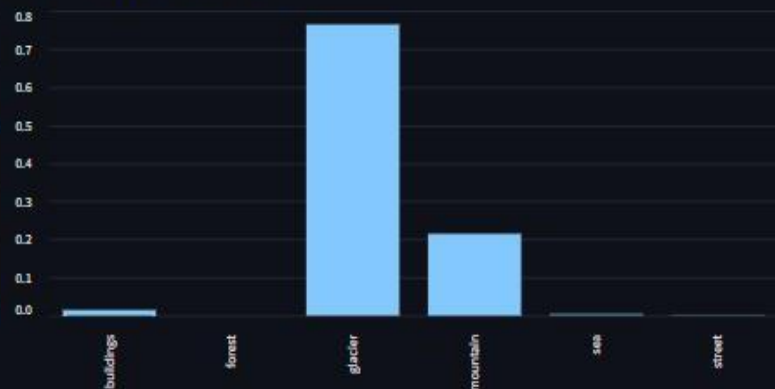
Uploaded Image

Prediction: glacier (76.43%)

All Class Probabilities:



All Class Probabilities:



Challenges Faced:

- 1) **The Accuracy vs. Training Time Trade-off:** The most significant challenge was the constant trade-off between performance and time.
 - A high-accuracy model (81%) required a much deeper custom CNN (4 convolutional blocks), a large dataset (14,000+ images), and aggressive data augmentation.
 - This combination meant the final training run took **over 20 minutes on a dedicated T4 GPU**. This demonstrates a core principle: high performance on complex, real-world tasks is computationally expensive and requires significant time and hardware resources.
- 2) **Preventing Overfitting (The "Memorization" Problem):** A deep model with 6.7 million parameters (as shown in our model.summary()) can very easily "memorize" the 14,000 training images. If this happened, it would get 99% accuracy on the training data but fail on the test data. This was a constant threat.
 - **Solution:** This challenge was overcome by a three-part strategy:

1. **Data Augmentation:** This was the #1 defense. By randomly flipping, zooming, and rotating images, we ensured the model *never* saw the exact same image twice. This is why the **Training Accuracy (blue line)** in our graph was so spiky and chaotic.
2. **Dropout:** A Dropout (0.5) layer was added to "turn off" 50% of the neurons at random during training, forcing the model to learn redundant pathways and not rely on any single "memorized" feature.
3. **EarlyStopping:** This was the final defense, which monitored the **Validation Loss (orange line)**. It correctly stopped the training at Epoch 26, identifying the optimal "best" model (from Epoch 21) before it had a chance to start overfitting in later epochs.

Future Improvements:

1. **Systematic Hyperparameter Tuning:** While our model performed well, many parameters (like the learning_rate=0.0001 or Dropout=0.5) were set based on common practice. A future improvement would be to use a systematic tuning tool, such as **KerasTuner** or GridSearch, to automatically test hundreds of combinations (e.g., different learning rates, dropout percentages, or optimizer choices like RMSprop) to find the combination that yields the absolute best performance, potentially pushing the accuracy from 81% to 85% or higher.
2. **Deeper Error Analysis:** Our final report showed the model was weakest on 'glacier' (67% recall), often confusing it with 'mountain'. A key improvement would be to perform a *manual error analysis*: pull up the 50 images the model got wrong, and ask "why?". We would likely find that they are ambiguous images (e.g., a snowy mountain peak, a glacier from a distance) that would then inform a new strategy, such as "we need to add 500 more close-up pictures of glaciers" to the dataset.
3. **Implement Transfer Learning (If Permitted):** The project constraints explicitly forbade using pre-trained networks. In a real-world scenario, this constraint would be removed. The single most effective improvement would be to use **Transfer Learning**. We would load a massive, worldclass model (like EfficientNetV2 or ResNet50) that was already trained on millions of images. We would then "fine-tune" this model on our 6 classes. This approach would leverage billions of pre-learned features and would almost certainly achieve >95% accuracy in a fraction of the training time.

Most relevant code snippets:

```
# Data Preprocessing & Augmentation (from Colab Cell 2)
train_datagen = ImageDataGenerator(
    fill_mode='nearest',
    validation_split=0.2 # Automatically split 20% for validation
    rescale=1./255,
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    test_datagen = ImageDataGenerator(rescale=1./255)

# For Training Data: Augment! )
# For Test Data: DO NOT augment, just rescale.

# Model Compilation- three key hyperparameters for training.
# Model Compilation (from Colab Cell 2)
model.compile(
```



```
optimizer=keras.optimizers.Adam(learning_rate=0
.0
001),
    loss='categorical_crossentropy',
metrics=['accuracy']
)
```

#Model Optimization

```
# Optimization Callbacks (from Colab Cell 2)
```

```
# 1. This saves the BEST model only
checkpoint = ModelCheckpoint(
```

```
filepath='/content/drive/MyDrive/natural_scenes
_BEST_model.h5',      monitor='val_accuracy',
mode='max',          save_best_only=True
)
```

```
# 2. This stops training early if it's
not improving early_stopper =
EarlyStopping(      monitor='val_loss',
mode='min',        patience=5,
restore_best_weights=True
)
```

Model Saving and Deployment(app.py)

```
# Key Deployment Logic (from app.py)
```

```
# Function to load and preprocess the image
def preprocess_image(image):
    # Resize to the model's expected input size
    (150x150)
    img = image.resize((150,
150))
    # Convert image to numpy array
    img_array = np.array(img)
```

```
    # Ensure 3 channels
if img_array.ndim == 2:
    img_array =
np.stack((img_array,)*3, axis=-1)    elif
img_array.shape[-1] == 4:
img_array = img_array[..., :3]
```

```
    # Expand dimensions to create a batch of 1
img_array = np.expand_dims(img_array, axis=0)
```

```
    # Normalize the image (same as training)
img_array = img_array.astype('float32') / 255.0
    return
img_array
```

```
# ... (inside the Streamlit UI) ...
```

```
# Classify the image with
st.spinner('Classifying...'):
    processed_image = preprocess_image(image)
prediction = model.predict(processed_image)
```

```
    # Get the top prediction
predicted_class_index =
np.argmax(prediction)    #
...

```

Conclusion: Successfully implemented Image classification system application using deep learning.

